UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

INFORME GENERAL – APE 4

Integración de clases, objetos y métodos (Java y C++)

Dato	Detalle
Asignatura	Algoritmos y Lógica de Programación
Nivel y paralelo	Primero – B
Estudiante	Paredes Acosta Dereck Shair
Docente	Ing. José Ruben Caizabuano, M.Sc.
Modalidad	Presencial
Ejercicios	5 (Libro y Autor, Rectángulo, Producto y Venta, Transferencias, Biblioteca)

1. Objetivos

Objetivo general

Integrar los conceptos de clases, objetos, atributos y métodos de la programación orientada a objetos mediante la implementación de cinco ejercicios prácticos en Java y C++, verificando su funcionamiento con pruebas de escritorio.

Objetivos específicos

1. Diseñar e implementar clases con atributos, constructores y métodos con distintos tipos de retorno (void, double, boolean y String).

2. Aplicar la colaboración entre objetos, empleando objetos como atributos y como parámetros de métodos en los casos Libro–Autor, Producto–Venta, Cuenta–Cuenta y Usuario–Libro–Prestamo.

3. Validar el funcionamiento de cada programa mediante pruebas de escritorio y la ejecución comparativa de las versiones en Java y C++.

2. Resumen de los ejercicios

N.º	Ejercicio	Concepto aplicado	Clases
1	Libro y Autor	Objeto como atributo	Autor, Libro
2	Rectángulo	Firmas de métodos	Rectangulo
3	Producto y Venta	Objeto como parámetro	Producto, Venta
4	Transferencias	Objeto como parámetro	Cuenta
5	Biblioteca	Colaboración entre clases	Usuario, Libro, Prestamo

3. Equipos y materiales

• Computadora personal con sistema operativo de escritorio.
• Java Development Kit (JDK) para compilar y ejecutar los programas .java.
• Compilador g++ (GCC) para compilar y ejecutar los programas .cpp.
• Editor de código o IDE (NetBeans, IntelliJ IDEA, Visual Studio Code o similar).
• Presentación APE4_Ejercicios_Grupo3.pptx con los enunciados de los ejercicios.
• Procesador de texto (Microsoft Word) para elaborar el informe.

4. Desarrollo de los ejercicios

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

Ejercicio 1: Libro y Autor (Objeto como atributo)

Enunciado

Crea la clase Autor (nombre, nacionalidad) y la clase Libro (titulo, anio, autor). Libro debe tener un atributo de tipo Autor y un método mostrarInformacion() que imprima el título, el año y el nombre del autor. En main, crea 2 libros.

Diseño de clases

Clase	Atributos	Métodos
Autor	nombre: String, nacionalidad: String	Constructor Autor(nombre, nacionalidad)
Libro	titulo: String, anio: int, autor: Autor	Constructor Libro(titulo, anio, autor); void mostrarInformacion()

Código Java (Ejercicio1.java)

```
// Ejercicio 1: Libro y Autor (objeto como atributo)
class Autor {
    String nombre;
    String nacionalidad;

    Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }
}

class Libro {
    String titulo;
    int anio;
    Autor autor; // objeto como atributo

    Libro(String titulo, int anio, Autor autor) {
        this.titulo = titulo;
        this.anio = anio;
        this.autor = autor;
    }

    void mostrarInformacion() {
        System.out.println("Título: " + titulo);
        System.out.println("Año: " + anio);
        System.out.println("Autor: " + autor.nombre + " (" + autor.nacionalidad + ")");
        System.out.println("-----------------------------");
    }
}

public class Ejercicio1 {
    public static void main(String[] args) {
        Autor a1 = new Autor("Gabriel García Márquez", "Colombiana");
        Autor a2 = new Autor("Jorge Icaza", "Ecuatoriana");

        Libro l1 = new Libro("Cien años de soledad", 1967, a1);
        Libro l2 = new Libro("Huasipungo", 1934, a2);

        l1.mostrarInformacion();
        l2.mostrarInformacion();
    }
}
```

Código C++ (Ejercicio1.cpp)

```
// Ejercicio 1: Libro y Autor (objeto como atributo)
#include <iostream>
#include <string>
using namespace std;

class Autor {
```

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

```
public:
    string nombre;
    string nacionalidad;

    Autor(string nombre, string nacionalidad) {
        this->nombre = nombre;
        this->nacionalidad = nacionalidad;
    }
};

class Libro {
public:
    string titulo;
    int anio;
    Autor autor; // objeto como atributo

    Libro(string titulo, int anio, Autor autor) : autor(autor) {
        this->titulo = titulo;
        this->anio = anio;
    }

    void mostrarInformacion() {
        cout << "Título: " << titulo << endl;
        cout << "Año: " << anio << endl;
        cout << "Autor: " << autor.nombre << " (" << autor.nacionalidad << ")" << endl;
        cout << "------------------------------" << endl;
    }
};

int main() {
    Autor a1("Gabriel García Márquez", "Colombiana");
    Autor a2("Jorge Icaza", "Ecuatoriana");

    Libro l1("Cien años de soledad", 1967, a1);
    Libro l2("Huasipungo", 1934, a2);

    l1.mostrarInformacion();
    l2.mostrarInformacion();
    return 0;
}
```

Prueba de escritorio

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Autor a1 = new Autor("Gabriel García Márquez", "Colombiana")	a1.nombre = "Gabriel García Márquez"; a1.nacionalidad = "Colombiana"	—
2	Autor a2 = new Autor("Jorge Icaza", "Ecuatoriana")	a2.nombre = "Jorge Icaza"; a2.nacionalidad = "Ecuatoriana"	—
3	Libro l1 = new Libro("Cien años de soledad", 1967, a1)	l1.titulo = "Cien años de soledad"; l1.anio = 1967; l1.autor → a1	—
4	Libro l2 = new Libro("Huasipungo", 1934, a2)	l2.titulo = "Huasipungo"; l2.anio = 1934; l2.autor → a2	—
5	l1.mostrarInformacion()	Lee l1.titulo, l1.anio y l1.autor.nombre (= a1.nombre)	Título: Cien años de soledad / Año: 1967 / Autor: Gabriel García Márquez (Colombiana)
6	l2.mostrarInformacion()	Lee l2.titulo, l2.anio y l2.autor.nombre (= a2.nombre)	Título: Huasipungo / Año: 1934 / Autor: Jorge Icaza (Ecuatoriana)

Salida obtenida (idéntica en Java y C++)

```
Título: Cien años de soledad
Año: 1967
Autor: Gabriel García Márquez (Colombiana)
------------------------------
Título: Huasipungo
Año: 1934
Autor: Jorge Icaza (Ecuatoriana)
```

```
----------------------------
```

Conclusión: Un objeto puede ser atributo de otra clase: Libro accede a los datos de Autor mediante autor.nombre sin duplicar información.

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

Ejercicio 2: Rectángulo (Firmas de métodos)

Enunciado

Crea la clase Rectangulo (base, altura) con los métodos calcularArea(), calcularPerimetro(), esCuadrado(), obtenerDescripcion() y mostrar(), usando el tipo de retorno correcto en cada firma. obtenerDescripcion() devuelve, por ejemplo, “Rectángulo de 4.0 x 6.0”, y mostrar() imprime todos los resultados.

Diseño de clases

Clase	Atributos	Métodos
Rectangulo	base: double, altura: double	double calcularArea(); double calcularPerimetro(); boolean esCuadrado(); String obtenerDescripcion(); void mostrar()

Código Java (Ejercicio2.java)

```
// Ejercicio 2: Rectángulo (firmas de métodos con distintos tipos de retorno)
class Rectangulo {
    double base;
    double altura;

    Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    double calcularArea() {
        return base * altura;
    }

    double calcularPerimetro() {
        return 2 * (base + altura);
    }

    boolean esCuadrado() {
        return base == altura;
    }

    String obtenerDescripcion() {
        return "Rectángulo de " + base + " x " + altura;
    }

    void mostrar() {
        System.out.println(obtenerDescripcion());
        System.out.println("Área: " + calcularArea());
        System.out.println("Perímetro: " + calcularPerimetro());
        System.out.println("¿Es cuadrado?: " + (esCuadrado() ? "Sí" : "No"));
        System.out.println("------------------------------");
    }
}

public class Ejercicio2 {
    public static void main(String[] args) {
        Rectangulo r1 = new Rectangulo(4, 6);
        Rectangulo r2 = new Rectangulo(5, 5);

        r1.mostrar();
        r2.mostrar();
    }
}
```

Código C++ (Ejercicio2.cpp)

```
// Ejercicio 2: Rectángulo (firmas de métodos con distintos tipos de retorno)
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
```

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

```
using namespace std;

class Rectangulo {
public:
    double base;
    double altura;

    Rectangulo(double base, double altura) {
        this->base = base;
        this->altura = altura;
    }

    double calcularArea() {
        return base * altura;
    }

    double calcularPerimetro() {
        return 2 * (base + altura);
    }

    bool esCuadrado() {
        return base == altura;
    }

    string obtenerDescripcion() {
        ostringstream ss;
        ss << fixed << setprecision(1) << "Rectángulo de " << base << " x " << altura;
        return ss.str();
    }

    void mostrar() {
        cout << fixed << setprecision(1);
        cout << obtenerDescripcion() << endl;
        cout << "Área: " << calcularArea() << endl;
        cout << "Perímetro: " << calcularPerimetro() << endl;
        cout << "¿Es cuadrado?: " << (esCuadrado() ? "Sí" : "No") << endl;
        cout << "------------------------------" << endl;
    }
};

int main() {
    Rectangulo r1(4, 6);
    Rectangulo r2(5, 5);

    r1.mostrar();
    r2.mostrar();
    return 0;
}
```

Prueba de escritorio

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Rectangulo r1 = new Rectangulo(4, 6)	r1.base = 4.0; r1.altura = 6.0	—
2	r1.mostrar() → obtenerDescripcion()	Concatena "Rectángulo de " + 4.0 + " x " + 6.0	Rectángulo de 4.0 x 6.0
3	r1.mostrar() → calcularArea()	4.0 × 6.0 = 24.0	Área: 24.0
4	r1.mostrar() → calcularPerimetro()	2 × (4.0 + 6.0) = 20.0	Perímetro: 20.0
5	r1.mostrar() → esCuadrado()	4.0 == 6.0 → false	¿Es cuadrado?: No
6	Rectangulo r2 = new Rectangulo(5, 5)	r2.base = 5.0; r2.altura = 5.0	—
7	r2.mostrar()	Descripción; área = 5.0 × 5.0 = 25.0; perímetro = 2 × (5.0 + 5.0) = 20.0; 5.0 == 5.0 → true	Rectángulo de 5.0 x 5.0 / Área: 25.0 / Perímetro: 20.0 / ¿Es cuadrado?: Sí

Salida obtenida (idéntica en Java y C++)

```
Rectángulo de 4.0 x 6.0
Área: 24.0
Perímetro: 20.0
¿Es cuadrado?: No
------------------------------
Rectángulo de 5.0 x 5.0
Área: 25.0
Perímetro: 20.0
¿Es cuadrado?: Sí
------------------------------
```

Conclusión: Elegir el tipo de retorno correcto (double, boolean, String, void) según lo que cada método debe entregar hace el código claro y reutilizable.

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

Ejercicio 3: Producto y Venta (Objeto como parámetro)

Enunciado

Crea la clase Producto (nombre, precio, stock) con reducirStock(int cantidad), que solo actúa si hay stock suficiente (si no, imprime "Stock insuficiente."), y mostrarProducto(). Crea la clase Venta (cantidad) con aplicar(Producto producto), que reduzca el stock y muestre el total a pagar.

Diseño de clases

Clase	Atributos	Métodos
Producto	nombre: String, precio: double, stock: int	boolean reducirStock(int cantidad); void mostrarProducto()
Venta	cantidad: int	void aplicar(Producto producto)

Código Java (Ejercicio3.java)

```
import java.util.Locale;

// Ejercicio 3: Producto y Venta (objeto como parámetro)
class Producto {
    String nombre;
    double precio;
    int stock;

    Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    // Solo actúa si hay stock suficiente; devuelve si la operación se realizó
    boolean reducirStock(int cantidad) {
        if (cantidad <= stock) {
            stock -= cantidad;
            return true;
        }
        System.out.println("Stock insuficiente.");
        return false;
    }

    void mostrarProducto() {
        System.out.printf(Locale.US, "Producto: %s | Precio: $%.2f | Stock: %d%n", nombre,
precio, stock);
    }
}

class Venta {
    int cantidad;

    Venta(int cantidad) {
        this.cantidad = cantidad;
    }

    // Recibe un objeto Producto como parámetro
    void aplicar(Producto producto) {
        System.out.println("Venta de " + cantidad + " unidad(es) de " + producto.nombre);
        if (producto.reducirStock(cantidad)) {
            double total = cantidad * producto.precio;
            System.out.printf(Locale.US, "Total a pagar: $%.2f%n", total);
        }
    }
}

public class Ejercicio3 {
    public static void main(String[] args) {
        Producto p = new Producto("Laptop", 850.50, 10);
        p.mostrarProducto();
```

```
        Venta v1 = new Venta(3);   // válida
        v1.aplicar(p);
        p.mostrarProducto();

        Venta v2 = new Venta(20);  // supera el stock
        v2.aplicar(p);
        p.mostrarProducto();
    }
}
```

Código C++ (Ejercicio3.cpp)

```
// Ejercicio 3: Producto y Venta (objeto como parámetro)
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Producto {
public:
    string nombre;
    double precio;
    int stock;

    Producto(string nombre, double precio, int stock) {
        this->nombre = nombre;
        this->precio = precio;
        this->stock = stock;
    }

    // Solo actúa si hay stock suficiente; devuelve si la operación se realizó
    bool reducirStock(int cantidad) {
        if (cantidad <= stock) {
            stock -= cantidad;
            return true;
        }
        cout << "Stock insuficiente." << endl;
        return false;
    }

    void mostrarProducto() {
        cout << fixed << setprecision(2);
        cout << "Producto: " << nombre << " | Precio: $" << precio
             << " | Stock: " << stock << endl;
    }
};

class Venta {
public:
    int cantidad;

    Venta(int cantidad) {
        this->cantidad = cantidad;
    }

    // Recibe un objeto Producto como parámetro (por referencia para modificar el original)
    void aplicar(Producto &producto) {
        cout << "Venta de " << cantidad << " unidad(es) de " << producto.nombre << endl;
        if (producto.reducirStock(cantidad)) {
            double total = cantidad * producto.precio;
            cout << fixed << setprecision(2) << "Total a pagar: $" << total << endl;
        }
    }
};

int main() {
    Producto p("Laptop", 850.50, 10);
    p.mostrarProducto();

    Venta v1(3);   // válida
    v1.aplicar(p);
    p.mostrarProducto();
```

```
    Venta v2(20);  // supera el stock
    v2.aplicar(p);
    p.mostrarProducto();
    return 0;
}
```

Prueba de escritorio

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Producto p = new Producto("Laptop", 850.50, 10)	p.nombre = "Laptop"; p.precio = 850.50; p.stock = 10	—
2	p.mostrarProducto()	Lee los atributos de p	Producto: Laptop | Precio: $850.50 | Stock: 10
3	Venta v1 = new Venta(3)	v1.cantidad = 3	—
4	v1.aplicar(p) → p.reducirStock(3)	3 <= 10 (verdadero) → p.stock = 10 − 3 = 7; retorna true	Venta de 3 unidad(es) de Laptop
5	v1.aplicar(p) → total	total = 3 × 850.50 = 2551.50	Total a pagar: $2551.50
6	p.mostrarProducto()	p.stock = 7	Producto: Laptop | Precio: $850.50 | Stock: 7
7	Venta v2 = new Venta(20); v2.aplicar(p) → p.reducirStock(20)	20 <= 7 (falso) → stock no cambia; retorna false	Venta de 20 unidad(es) de Laptop / Stock insuficiente.
8	p.mostrarProducto()	p.stock = 7 (sin cambios)	Producto: Laptop | Precio: $850.50 | Stock: 7

Salida obtenida (idéntica en Java y C++)

```
Producto: Laptop | Precio: $850.50 | Stock: 10
Venta de 3 unidad(es) de Laptop
Total a pagar: $2551.50
Producto: Laptop | Precio: $850.50 | Stock: 7
Venta de 20 unidad(es) de Laptop
Stock insuficiente.
Producto: Laptop | Precio: $850.50 | Stock: 7
```

Conclusión: Al pasar el objeto Producto como parámetro, Venta modifica el stock del objeto original; la validación evita ventas que superen las existencias.

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

Ejercicio 4: Transferencias (Objeto como parámetro)

Enunciado

Usa la clase Cuenta y agrega el método transferir(Cuenta destino, double monto), que retire el monto de la cuenta actual y lo deposite en la cuenta destino solo si hay fondos. En main, crea 2 cuentas, haz una transferencia válida y otra que falle.

Diseño de clases

Clase	Atributos	Métodos
Cuenta	titular: String, saldo: double	void depositar(double monto); boolean retirar(double monto); void transferir(Cuenta destino, double monto); void mostrar()

Código Java (Ejercicio4.java)

```
import java.util.Locale;

// Ejercicio 4: Transferencias entre cuentas (objeto como parámetro)
class Cuenta {
    String titular;
    double saldo;

    Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    void depositar(double monto) {
        saldo += monto;
    }

    boolean retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            return true;
        }
        return false;
    }

    // Retira de esta cuenta y deposita en la cuenta destino solo si hay fondos
    void transferir(Cuenta destino, double monto) {
        if (retirar(monto)) {
            destino.depositar(monto);
            System.out.printf(Locale.US, "Transferencia exitosa: $%.2f de %s a %s%n", monto,
titular, destino.titular);
        } else {
            System.out.printf(Locale.US, "Transferencia fallida: fondos insuficientes en la
cuenta de %s%n", titular);
        }
    }

    void mostrar() {
        System.out.printf(Locale.US, "Cuenta de %s -> Saldo: $%.2f%n", titular, saldo);
    }
}

public class Ejercicio4 {
    public static void main(String[] args) {
        Cuenta c1 = new Cuenta("Ana", 500);
        Cuenta c2 = new Cuenta("Luis", 200);

        System.out.println("Saldos iniciales:");
        c1.mostrar();
        c2.mostrar();

        System.out.println("\nTransferencia válida (150):");
        c1.transferir(c2, 150);
```

```
        c1.mostrar();
        c2.mostrar();

        System.out.println("\nTransferencia que falla (1000):");
        c1.transferir(c2, 1000);
        c1.mostrar();
        c2.mostrar();
    }
}
```

Código C++ (Ejercicio4.cpp)

```
// Ejercicio 4: Transferencias entre cuentas (objeto como parámetro)
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Cuenta {
public:
    string titular;
    double saldo;

    Cuenta(string titular, double saldo) {
        this->titular = titular;
        this->saldo = saldo;
    }

    void depositar(double monto) {
        saldo += monto;
    }

    bool retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            return true;
        }
        return false;
    }

    // Retira de esta cuenta y deposita en la cuenta destino solo si hay fondos
    void transferir(Cuenta &destino, double monto) {
        cout << fixed << setprecision(2);
        if (retirar(monto)) {
            destino.depositar(monto);
            cout << "Transferencia exitosa: $" << monto << " de " << titular
                 << " a " << destino.titular << endl;
        } else {
            cout << "Transferencia fallida: fondos insuficientes en la cuenta de "
                 << titular << endl;
        }
    }

    void mostrar() {
        cout << fixed << setprecision(2);
        cout << "Cuenta de " << titular << " -> Saldo: $" << saldo << endl;
    }
};

int main() {
    Cuenta c1("Ana", 500);
    Cuenta c2("Luis", 200);

    cout << "Saldos iniciales:" << endl;
    c1.mostrar();
    c2.mostrar();

    cout << "\nTransferencia válida (150):" << endl;
    c1.transferir(c2, 150);
    c1.mostrar();
    c2.mostrar();
```

```
    cout << "\nTransferencia que falla (1000):" << endl;
    c1.transferir(c2, 1000);
    c1.mostrar();
    c2.mostrar();
    return 0;
}
```

Prueba de escritorio

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Cuenta c1 = new Cuenta("Ana", 500)	c1.titular = "Ana"; c1.saldo = 500.00	—
2	Cuenta c2 = new Cuenta("Luis", 200)	c2.titular = "Luis"; c2.saldo = 200.00	—
3	c1.mostrar(); c2.mostrar()	Lee los saldos	Saldo Ana: $500.00 / Saldo Luis: $200.00
4	c1.transferir(c2, 150) → c1.retirar(150)	150 > 0 y 150 <= 500 (verdadero) → c1.saldo = 500 − 150 = 350.00	—
5	destino.depositar(150)	c2.saldo = 200 + 150 = 350.00	Transferencia exitosa: $150.00 de Ana a Luis
6	c1.mostrar(); c2.mostrar()	c1.saldo = 350.00; c2.saldo = 350.00	Saldo Ana: $350.00 / Saldo Luis: $350.00
7	c1.transferir(c2, 1000) → c1.retirar(1000)	1000 <= 350 (falso) → no se retira; c2 no cambia	Transferencia fallida: fondos insuficientes en la cuenta de Ana
8	c1.mostrar(); c2.mostrar()	c1.saldo = 350.00; c2.saldo = 350.00 (sin cambios)	Saldo Ana: $350.00 / Saldo Luis: $350.00

Salida obtenida (idéntica en Java y C++)

```
Saldos iniciales:
Cuenta de Ana -> Saldo: $500.00
Cuenta de Luis -> Saldo: $200.00

Transferencia válida (150):
Transferencia exitosa: $150.00 de Ana a Luis
Cuenta de Ana -> Saldo: $350.00
Cuenta de Luis -> Saldo: $350.00

Transferencia que falla (1000):
Transferencia fallida: fondos insuficientes en la cuenta de Ana
Cuenta de Ana -> Saldo: $350.00
Cuenta de Luis -> Saldo: $350.00
```

Conclusión: Una transferencia solo es válida si el retiro se realiza; así los saldos de ambas cuentas permanecen consistentes cuando la operación falla.

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

Ejercicio 5: Biblioteca (Colaboración entre clases)

Enunciado

Crea tres clases que colaboren entre sí: Usuario, Libro y Prestamo. Prestamo tiene atributos de tipo Usuario y Libro, y su método registrar() intenta prestar el libro y muestra un mensaje según el resultado. En main, haz que dos usuarios intenten pedir el mismo libro.

Diseño de clases

Clase	Atributos	Métodos
Usuario	nombre: String, cedula: String	Constructor Usuario(nombre, cedula)
Libro	titulo: String, disponible: boolean	boolean prestar(); void devolver()
Prestamo	usuario: Usuario, libro: Libro	void registrar()

Código Java (Ejercicio5.java)

```
// Ejercicio 5: Biblioteca (colaboración entre clases)
class Usuario {
    String nombre;
    String cedula;

    Usuario(String nombre, String cedula) {
        this.nombre = nombre;
        this.cedula = cedula;
    }
}

class Libro {
    String titulo;
    boolean disponible;

    Libro(String titulo) {
        this.titulo = titulo;
        this.disponible = true;
    }

    // Intenta prestar el libro: solo es posible si está disponible
    boolean prestar() {
        if (disponible) {
            disponible = false;
            return true;
        }
        return false;
    }

    void devolver() {
        disponible = true;
    }
}

class Prestamo {
    Usuario usuario;
    Libro libro;

    Prestamo(Usuario usuario, Libro libro) {
        this.usuario = usuario;
        this.libro = libro;
    }

    void registrar() {
        if (libro.prestar()) {
            System.out.println("Préstamo exitoso: \"" + libro.titulo + "\" fue prestado a "
                    + usuario.nombre + " (C.I. " + usuario.cedula + ").");
        } else {
            System.out.println("Préstamo rechazado: \"" + libro.titulo + "\" no está disponible
para "
                    + usuario.nombre + " (C.I. " + usuario.cedula + ").");
        }
```

```
    }
}

public class Ejercicio5 {
    public static void main(String[] args) {
        Usuario u1 = new Usuario("Juan Pérez", "1805123456");
        Usuario u2 = new Usuario("María López", "1804987654");
        Libro libro = new Libro("Cien años de soledad");

        Prestamo p1 = new Prestamo(u1, libro);
        Prestamo p2 = new Prestamo(u2, libro);

        p1.registrar(); // el libro está disponible
        p2.registrar(); // el libro ya fue prestado
    }
}
```

Código C++ (Ejercicio5.cpp)

```
// Ejercicio 5: Biblioteca (colaboración entre clases)
#include <iostream>
#include <string>
using namespace std;

class Usuario {
public:
    string nombre;
    string cedula;

    Usuario(string nombre, string cedula) {
        this->nombre = nombre;
        this->cedula = cedula;
    }
};

class Libro {
public:
    string titulo;
    bool disponible;

    Libro(string titulo) {
        this->titulo = titulo;
        this->disponible = true;
    }

    // Intenta prestar el libro: solo es posible si está disponible
    bool prestar() {
        if (disponible) {
            disponible = false;
            return true;
        }
        return false;
    }

    void devolver() {
        disponible = true;
    }
};

class Prestamo {
public:
    Usuario *usuario;
    Libro *libro; // puntero para compartir el mismo libro entre préstamos

    Prestamo(Usuario *usuario, Libro *libro) {
        this->usuario = usuario;
        this->libro = libro;
    }

    void registrar() {
        if (libro->prestar()) {
```

```
            cout << "Préstamo exitoso: \"" << libro->titulo << "\" fue prestado a "
                 << usuario->nombre << " (C.I. " << usuario->cedula << ")." << endl;
        } else {
            cout << "Préstamo rechazado: \"" << libro->titulo << "\" no está disponible para "
                 << usuario->nombre << " (C.I. " << usuario->cedula << ")." << endl;
        }
    }
};

int main() {
    Usuario u1("Juan Pérez", "1805123456");
    Usuario u2("María López", "1804987654");
    Libro libro("Cien años de soledad");

    Prestamo p1(&u1, &libro);
    Prestamo p2(&u2, &libro);

    p1.registrar(); // el libro está disponible
    p2.registrar(); // el libro ya fue prestado
    return 0;
}
```

Prueba de escritorio

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Usuario u1 = new Usuario("Juan Pérez", "1805123456")	u1.nombre = "Juan Pérez"; u1.cedula = "1805123456"	—
2	Usuario u2 = new Usuario("María López", "1804987654")	u2.nombre = "María López"; u2.cedula = "1804987654"	—
3	Libro libro = new Libro("Cien años de soledad")	libro.titulo = "Cien años de soledad"; libro.disponible = true	—
4	Prestamo p1 = new Prestamo(u1, libro); Prestamo p2 = new Prestamo(u2, libro)	p1 → (u1, libro); p2 → (u2, libro). Ambos comparten el mismo objeto libro	—
5	p1.registrar() → libro.prestar()	libro.disponible es true → pasa a false; retorna true	Préstamo exitoso: "Cien años de soledad" fue prestado a Juan Pérez (C.I. 1805123456).
6	p2.registrar() → libro.prestar()	libro.disponible es false → no cambia; retorna false	Préstamo rechazado: "Cien años de soledad" no está disponible para María López (C.I. 1804987654).

Salida obtenida (idéntica en Java y C++)

```
Préstamo exitoso: "Cien años de soledad" fue prestado a Juan Pérez (C.I. 1805123456).
Préstamo rechazado: "Cien años de soledad" no está disponible para María López (C.I.
1804987654).
```

Conclusión: Dos préstamos comparten el mismo objeto Libro; por eso el segundo usuario recibe un rechazo: la colaboración entre clases mantiene un estado único.

5. Conclusiones

1. Se implementaron los cinco ejercicios en Java y C++, y ambas versiones produjeron salidas idénticas, lo que confirma que los conceptos de clase, objeto, atributo y método son los mismos en los dos lenguajes y solo cambia su sintaxis.

2. El uso de objetos como atributos (Libro–Autor, Prestamo–Usuario–Libro) y como parámetros (Venta–Producto, Cuenta–Cuenta) permite que las clases colaboren y modifiquen el estado de otros objetos sin duplicar datos.

3. Las validaciones dentro de los métodos (stock suficiente, fondos suficientes, libro disponible) y las pruebas de escritorio demostraron que el programa se comporta correctamente tanto en casos válidos como en casos de fallo.

6. Recomendaciones

1. Realizar la prueba de escritorio antes de ejecutar el código, siguiendo el valor de cada variable paso a paso, para detectar errores de lógica a tiempo.

2. Definir con claridad el tipo de retorno de cada método (void, boolean, double, String) y hacer que los métodos devuelvan si la operación se cumplió, de modo que otras clases puedan reaccionar al resultado.

3. Al pasar de Java a C++, tener presente la diferencia en el manejo de objetos: en C++ conviene usar referencias o punteros cuando el método debe modificar el objeto original, como en Venta.aplicar() y Cuenta.transferir().